Cuándo Florece

Arquitectura técnica y diagramas C4 · Reto Greenia · Hackathon EPS UFV, octubre de 2026
Prototipo en vivo: https://cuandoflorece.cabecerastudio.com

1. Resumen

Cuándo Florece es un mapa de floración para España: dice a un viajero adónde ir, cuándo y con qué confianza, y da a cada destino un panel con la demanda prevista y el control de su visibilidad y aforo. La arquitectura sigue cuatro principios que vienen del propio reto:

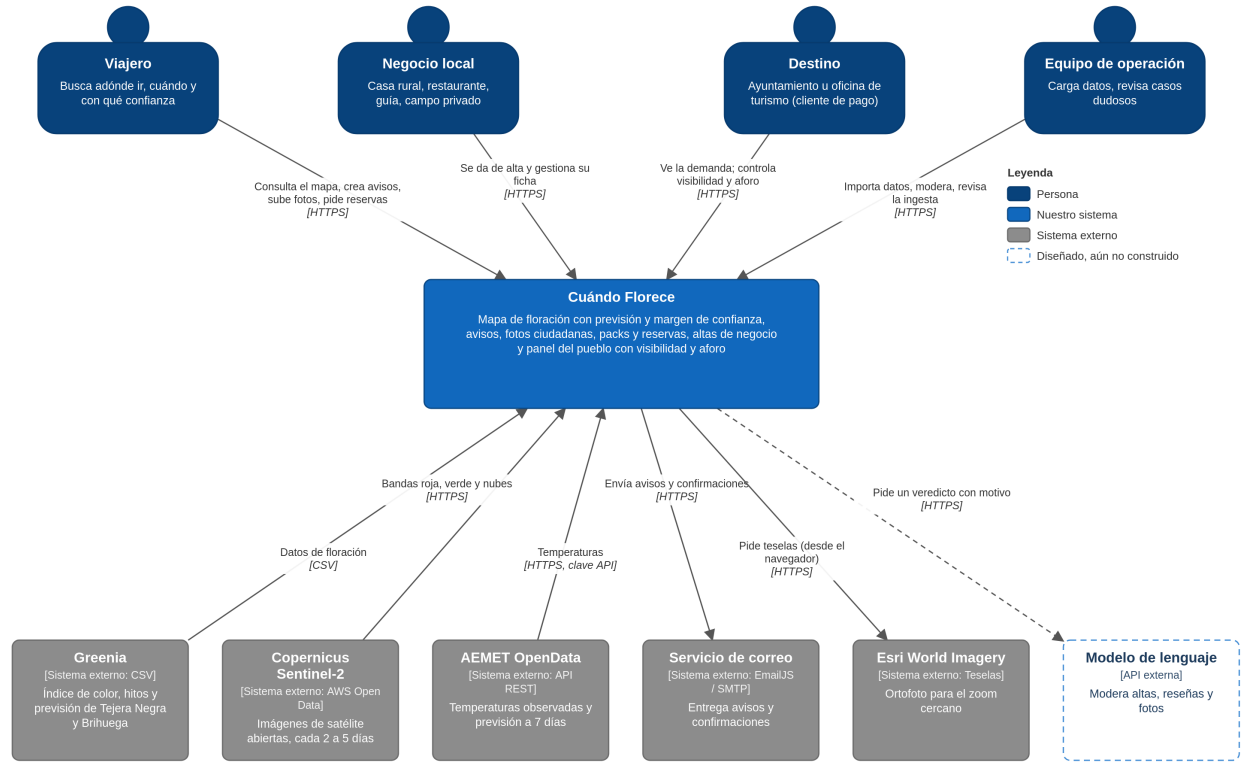
- **Funciona solo.** Cada mañana una tarea programada descarga satélite y temperaturas, recalcula el estado y la previsión de cada destino y envía los avisos. Nadie revisa datos a mano.
- **Rangos, no fechas falsas.** Toda previsión es una ventana (desde, hasta) con su confianza y su probabilidad, y cada dato lleva su tipo (observado, estimado, previsto) y su fuente.
- **Lo comercial va aparte.** Alojamientos, packs y patrocinio están en un bloque separado de la respuesta y nunca cambian la previsión.
- **Un solo contrato.** La API publica un documento OpenAPI. La web genera sus tipos a partir de él, así que web y servidor no se pueden desincronizar.

Hay dos despliegues de la misma aplicación. El **diseño de producción** (sección 3.1) tiene API, base de datos geográfica y tareas programadas; está construido y probado con 25 tests. El **prototipo en vivo** (sección 3.2) ejecuta la misma interfaz con una API simulada dentro del navegador, para que la demo no dependa de un servidor de aplicación. Ambos usan los datos reales de Greenia.

2. Nivel 1: contexto

Nivel 1 · Diagrama de contexto

Cuándo Florece y las personas y sistemas con los que se relaciona



Elemento	Papel
Viajero	Usuario gratuito. Consulta el mapa y las fichas, crea avisos, sube fotos y pide reservas de packs.
Negocio local	Se da de alta para aparecer en la ficha del destino y en los avisos (ficha de pago por temporada).
Destino	Cliente principal (licencia anual). Ve la demanda agregada y decide su visibilidad y aforo.
Equipo de operación	Importa los datos de Greenia, revisa la ingesta y los casos dudosos de moderación.
Greenia	Datos de partida: índice de color por satélite en 7 zonas de Tejera Negra, temperaturas, hitos, previsión, y la temporada de la lavanda de Brihuega.
Copernicus Sentinel-2	Imágenes abiertas (nivel L2A, formato COG en AWS Open Data). De ellas sale el índice de color de cada zona.
AEMET OpenData	Temperaturas observadas y previsión municipal a 7 días.
Servicio de correo	Entrega avisos y confirmaciones. Hoy EmailJS en el prototipo; en producción, un proveedor SMTP.

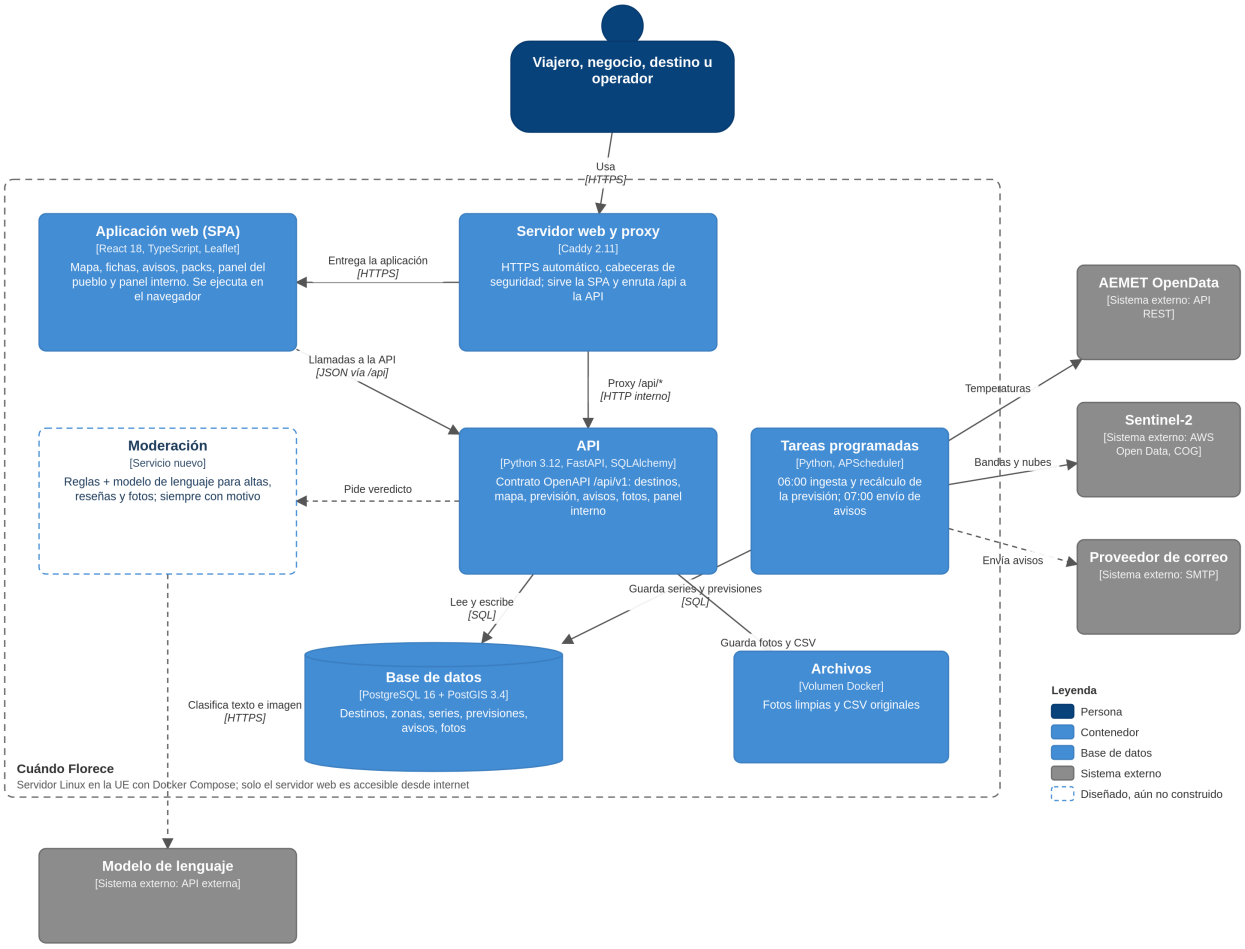
Esri World Imagery	Ortofoto para el zoom cercano del mapa.
Modelo de lenguaje (diseñado)	Clasifica altas, reseñas y fotos con un motivo. Aún no está construido (ver sección 8).

3. Nivel 2: contenedores

3.1 Diseño de producción

Nivel 2 · Contenedores (diseño de producción)

El backend está construido y probado (25 tests); el despliegue completo está en deploy/fullstack/

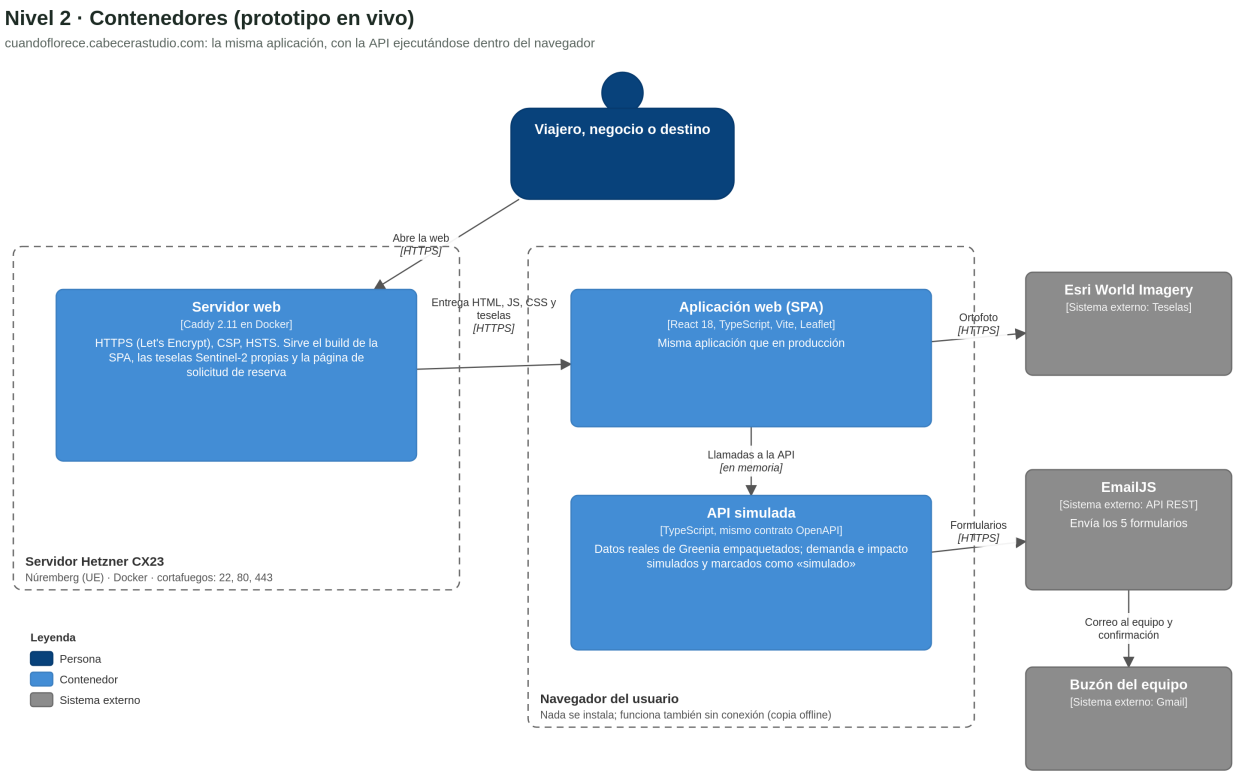


Contenedor	Tecnología	Responsabilidad
Aplicación web (SPA)	React 18, TypeScript, Vite, Leaflet	Mapa, fichas, avisos, packs, panel del pueblo y panel interno. Bilingüe (ES/EN).
Servidor web y proxy	Caddy 2.11	Único punto expuesto a internet. HTTPS automático, cabeceras de seguridad, sirve la SPA y enruta `/api/*` a la API.
API	Python 3.12, FastAPI,	Contrato `/api/v1` (documentación en

	SQLAlchemy, Alembic	`/api/v1/docs`): destinos, mapa por fecha, serie, previsión, avisos, fotos, sesión y panel interno.
Tareas programadas	Python, APScheduler (misma imagen que la API)	06:00 ingesta (AEMET y Sentinel-2) y recálculo; 07:00 envío de los avisos que tocan.
Base de datos	PostgreSQL 16 + PostGIS 3.4	21 tablas: destinos y zonas, series de índice y meteorología, ventanas y previsiones, avisos, fotos, contenidos, ofertas, registro de ingesta.
Archivos	Volumen Docker	Fotos ya limpias de metadatos y los CSV originales con su procedencia.
Moderación (diseñado)	Servicio nuevo + API de un modelo de lenguaje	Veredicto con motivo para altas, reseñas y fotos; revisión humana en los dudosos.

Todo se describe en un único archivo de Docker Compose (`deploy/fullstack/`). La base de datos, la API y las tareas solo están en la red interna; no publican puertos.

3.2 Prototipo en vivo

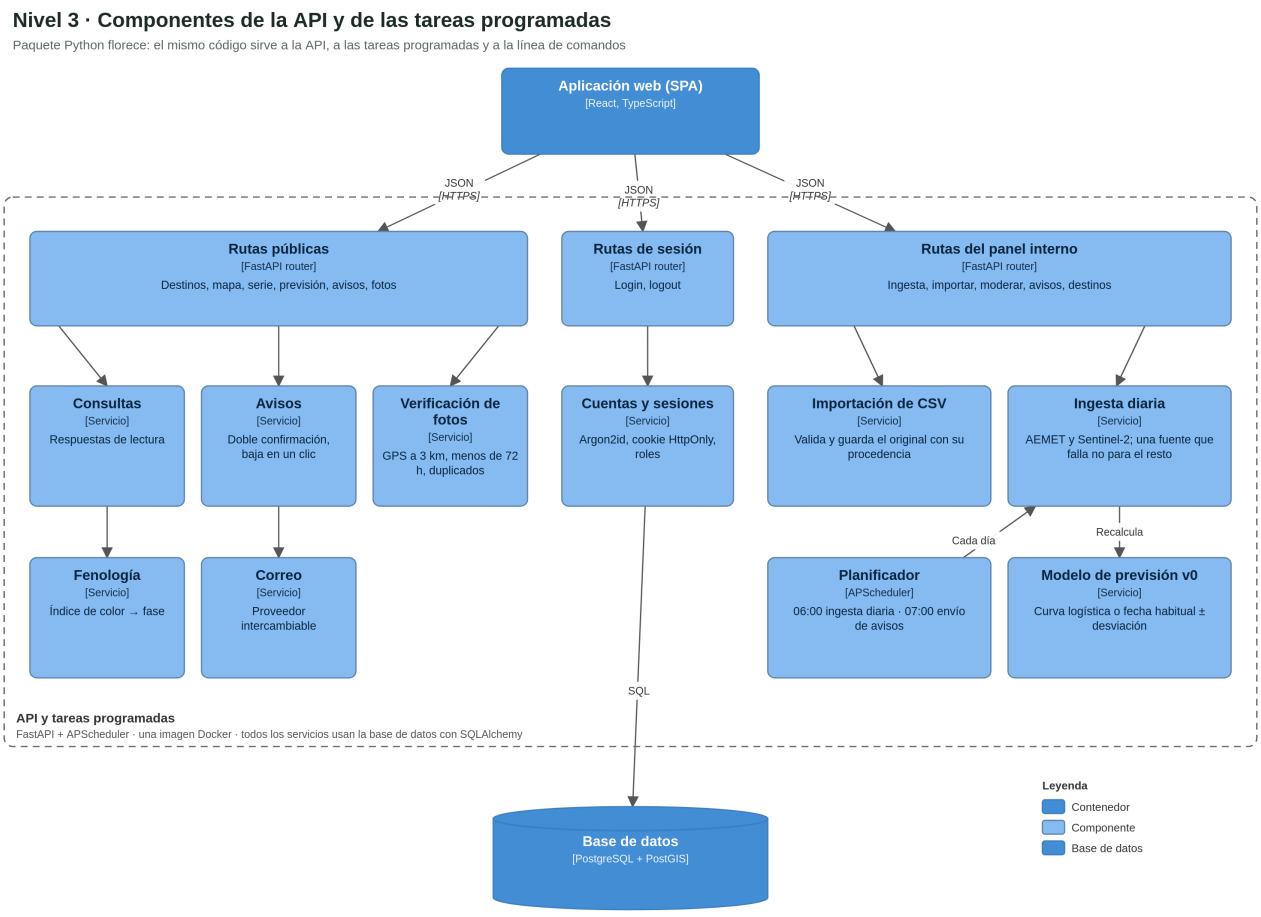


La web en vivo es la misma SPA compilada en modo demo (`VITE_API_MODE=mock`). En lugar de llamar a `/api/v1`, usa un adaptador en memoria que implementa la misma interfaz y devuelve los mismos tipos que la API real, con los datos reales de Greenia empaquetados. Lo sirve Caddy en un servidor Hetzner CX23 (2 vCPU, 4 GB, Núremberg, UE). Los cinco formularios (aviso, foto, alta de negocio, contacto para licencia, solicitud de reserva) envían correo real al equipo a través de EmailJS, y la persona recibe una

confirmación.

Aspecto	Prototipo en vivo	Producción
API	Simulada en el navegador, mismo contrato	FastAPI + PostgreSQL
Datos de Greenia	Reales, empaquetados en el build	Reales, importados por CSV
Satélite y temperaturas	Teselas Sentinel-2 precalculadas	Ingesta diaria automática
Demanda e impacto del panel	Simulados y marcados «simulado»	Agregados de los avisos reales
Moderación	Casos preparados	Servicio de moderación (sección 8)
Correo	EmailJS desde el navegador	Proveedor SMTP desde el backend

4. Nivel 3: componentes de la API



El paquete Python `florece` separa rutas (qué se expone) de servicios (qué se hace). El mismo código sirve a la API, a las tareas programadas y a la línea de comandos, de modo que el botón «ejecutar ingesta» del panel y la tarea de las 06:00 hacen exactamente lo mismo.

Componente	Qué hace
Fenología	Convierte el índice de color de una zona en una fase (inicio, avance, pleno, final) con umbrales propios de cada destino, y da el estado de un destino en cualquier fecha.
Modelo de previsión v0	Si Greenia aporta ventanas, las publica tal cual. Si no, ajusta una curva logística a los datos de la temporada; sin datos suficientes, usa la fecha habitual ± la desviación histórica. El margen crece con la distancia a la fecha. Es pequeño y sustituible: un único punto de entrada.
Ingesta diaria	AEMET, después Sentinel-2, después recálculo. Una fuente que falla no detiene el resto: se conserva su último dato válido y la fuente queda marcada como desactualizada.
Importación de CSV	Valida y normaliza los datos de Greenia (fechas ISO, °C, índice de 0 a 1) y guarda el archivo original con su procedencia.
Avisos	Doble confirmación por correo; un aviso por temporada cuando la ventana entra en la antelación elegida o el satélite confirma la fase; baja en un clic (RFC 8058).
Verificación de fotos	Lee GPS y hora de la foto: se acepta a menos de 3 km de una zona y con menos de 72 h. Duplicados por huella SHA-256 y perceptual; máximo 5 fotos al día por persona. Se guarda solo una copia nueva sin metadatos.
Cuentas y sesiones	Contraseñas con Argon2id; sesión en cookie HttpOnly, Secure y SameSite=Strict; en la base solo se guarda el hash del token. Roles admin y editor.
Correo	Interfaz con un único método `enviar()`; cambiar de proveedor es añadir una clase. Hoy escribe en un registro y una tabla de salida que el panel muestra.

5. Cómo circula la información

5.1 Ingesta y previsión (cada día, 06:00)

1. Para cada destino, AEMET devuelve las temperaturas observadas y la previsión a 7 días.
2. Para cada escena Sentinel-2 reciente (hasta 10 días, nubosidad máxima del 60 %), se leen las bandas roja y verde a 10 m y la clasificación de escena alrededor de cada zona. Se descartan nubes, sombras, nieve y agua. Si queda menos del 30 % de píxeles útiles, la zona se marca «sin dato (nubes)»: los huecos nunca se rellenan.
3. El índice de color es la media de (R − G) / (R + G), calibrado de 0 a 1 por destino. Sube cuando las hojas amarillean o los campos florecen.
4. El modelo recalcula el estado actual y las ventanas de cada fase, y lo guarda.

5.2 Consulta del mapa

La SPA pide `/mapa?fecha=` y recibe el estado de todos los destinos para ese día: observado si es hoy, previsto con su confianza si es una fecha futura. La ficha (`/destinos/{id}`) devuelve cuatro bloques separados: destino, calculado, editorial (lo escribe el pueblo) y comercial.

5.3 Aviso «avísame cuando florezca»

Alta con email, destino, fase, antelación, provincia y consentimiento → correo de confirmación → aviso confirmado. A las 07:00 la tarea envía los que tocan. La provincia solo se usa agregada para el panel del pueblo («1.240 personas esperan la lavanda, el 58 % desde Madrid»).

5.4 Visibilidad y aforo del pueblo

El destino elige visibilidad normal, reducida o pausada y su aforo diario. Si la demanda prevista de un fin de semana supera el aforo, el mapa público muestra «aforo completo» y propone días entre semana y pueblos cercanos. En el prototipo esto funciona en la demo; en el backend son los próximos endpoints a construir, con los tipos ya definidos en la web.

6. Tecnologías y por qué

Pieza	Elegido	Por qué (y qué descartamos)
Interfaz	React 18 + TypeScript + Vite	Tipos generados desde el OpenAPI; el compilador detecta cualquier cambio de contrato. Vite compila en segundos. El build demo usa rutas relativas y funciona incluso sin conexión.
Mapa	Leaflet + teselas Sentinel-2 propias + Esri	Libre y ligero. Descartamos Google Maps y Mapbox por licencia y coste por visita.
API	FastAPI (Python 3.12)	Valida entradas y genera el OpenAPI solo. Python es el lenguaje natural para leer imágenes de satélite (lectura de COG) y ajustar curvas.
Base de datos	PostgreSQL + PostGIS	Consultas geográficas (zonas, radio de 3 km de las fotos) dentro de la base. Una base NoSQL no aporta nada aquí y complica las consultas por fecha.
Tareas	APScheduler	Dos tareas diarias no justifican una cola de mensajes; es la misma imagen que la API.
Servidor web	Caddy	Certificados HTTPS automáticos y cabeceras en un archivo de 30 líneas. Frente a Nginx + Certbot, menos piezas.
Despliegue	Docker Compose en un servidor en la UE	Un archivo describe todo; desplegar es «actualizar el repositorio y un comando». Kubernetes o serverless son

		excesivos para esta escala. Datos en la UE (RGPD).
Correo	EmailJS (prototipo), SMTP (producción)	EmailJS permite correo real sin backend en la demo. En producción el backend envía con doble confirmación y baja en un clic.

7. Seguridad y protección de datos

- **Red:** cortafuegos con solo los puertos 22, 80 y 443; acceso SSH solo con clave; solo Caddy publica puertos.
- **Transporte:** HTTPS obligatorio con HSTS. Política de seguridad de contenido (CSP) que solo permite los orígenes necesarios (fuentes, ortofoto, EmailJS); `frame-ancestors 'none'`, `nosniff`, Referrer-Policy y Permissions-Policy.
- **Cuentas:** Argon2id, cookie HttpOnly/Secure/SameSite=Strict, solo el hash del token en la base.
- **Datos personales mínimos:** un aviso guarda email, provincia (opcional) y consentimiento. Doble confirmación y baja en un clic. Los destinos solo ven cifras agregadas; no se venden datos.
- **Fotos:** se borran los metadatos (GPS, dispositivo) antes de guardarlas; límite diario por persona con IP anonimizada por hash.
- **Sin cookies de seguimiento** ni analítica de terceros.
- **Prototipo:** los formularios pasan por EmailJS, un servicio fuera de la UE, hasta el buzón del equipo. En producción el correo sale del backend.

8. Escalabilidad, fiabilidad y mantenimiento

- **Un destino nuevo es configuración, no código:** coordenadas de las zonas, especie, umbrales, estación de AEMET y municipio. El coste marginal es casi nulo.
- **Escala horizontal:** la API no guarda estado en memoria; se pueden añadir réplicas detrás de Caddy. La SPA es estática y puede ir a una CDN.
- **Carga baja y predecible:** la ingesta es una pasada diaria por destino; las consultas del mapa son lecturas cacheables.
- **Fiabilidad:** una fuente caída no para la ingesta; el panel interno muestra el estado de cada fuente y el registro de cada ejecución.
- **Copias de seguridad:** volcado diario de la base y de los archivos, con 14 días de retención (`deploy/fullstack/backup.sh`).
- **Mantenimiento:** migraciones de base de datos con Alembic; el modelo de previsión es un módulo sustituible; el contrato OpenAPI documenta cada cambio.
- **Moderación (siguiente pieza):** reglas deterministas (ubicación, fecha, duplicados) más un modelo de lenguaje que clasifica texto e imagen y devuelve siempre un motivo; los casos dudosos van a revisión humana en el panel.

9. Limitaciones, dependencias y riesgos

Limitación o riesgo	Cómo lo tratamos
El índice de color por satélite es una estimación, no una medida	Se dice en cada ficha y en «cómo calculamos esto»; las fotos ciudadanas lo contrastan.
Nubes sobre las zonas	La zona queda «sin dato» ese día; el estado se apoya en las demás zonas y en la temperatura.
Pocas temporadas de histórico	Modelo v0 explicable que mejora cada temporada; siempre con margen y confianza.
Dependencia de AEMET y Copernicus	Si fallan, se conserva el último dato válido y la fuente se marca desactualizada.
Endpoints del modelo de negocio (panel del pueblo, visibilidad, altas) solo existen en la demo	Sus tipos ya están definidos en la web; son el siguiente trabajo del backend.
Moderación con IA no construida	Diseñada (sección 8); en la demo se muestran casos preparados.
Plan gratuito de EmailJS: 200 correos al mes, sin adjuntos	Suficiente para la demo; en producción, proveedor SMTP desde el backend.

10. Pruebas y evidencias

- **Backend:** 25 tests automáticos (pytest) contra una base PostGIS real: 20 de la API, 4 del modelo de previsión y 1 del cliente de AEMET. Todos pasan.
- **Web:** el build comprueba los tipos de TypeScript contra el contrato OpenAPI; un error de contrato no compila.
- **Prototipo en vivo:** el guion de la demo recorrido de principio a fin sin errores en consola; los cinco formularios probados con entrega real de correo.
- **Datos:** los CSV de Greenia (Tejera Negra y Brihuega) están integrados; los otros destinos citan sus fuentes públicas en cada ficha.

Código: frontend/ (web), backend/ (API, tareas, tests), deploy/ (prototipo) y deploy/fullstack/ (producción). Contrato de la API: docs/API.md y docs/openapi.json.